



NUST CHIP DESIGN CENTRE

RISC-V Architecture

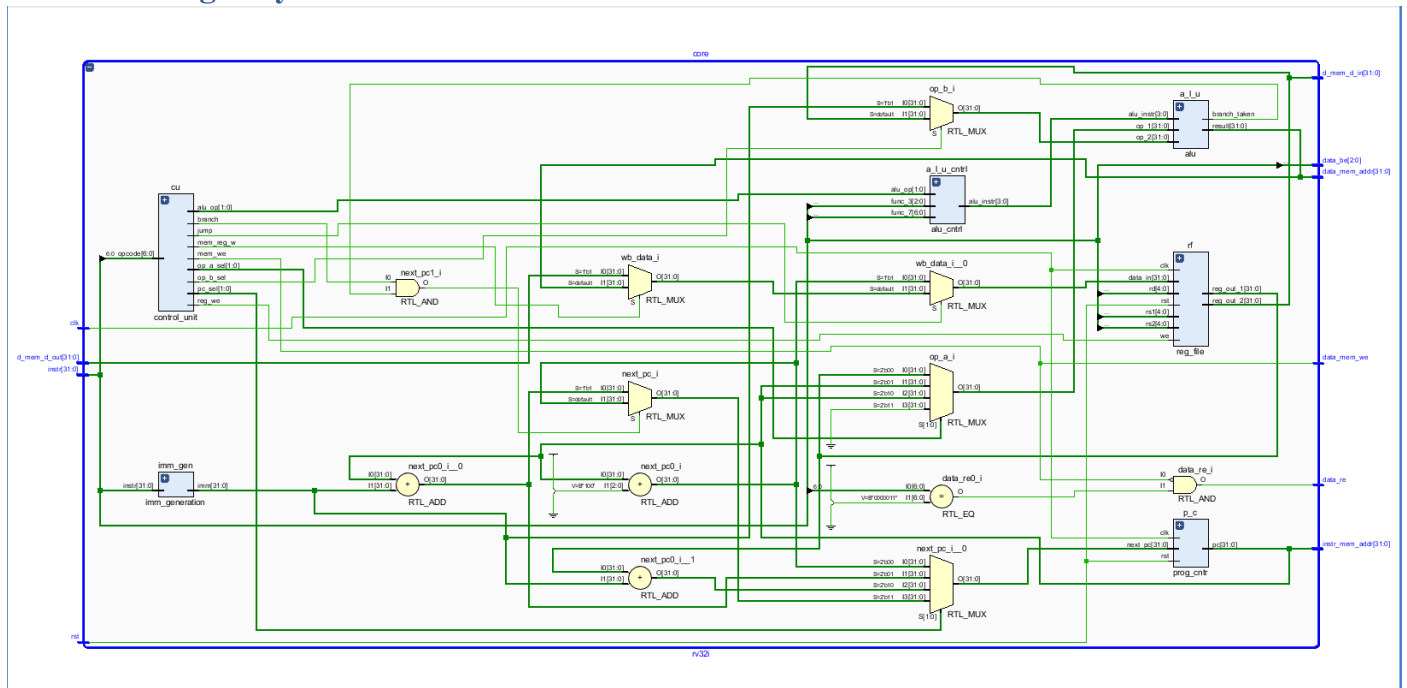
Weekly Task

RISC-V – UART Integration and Digital Clock Testing

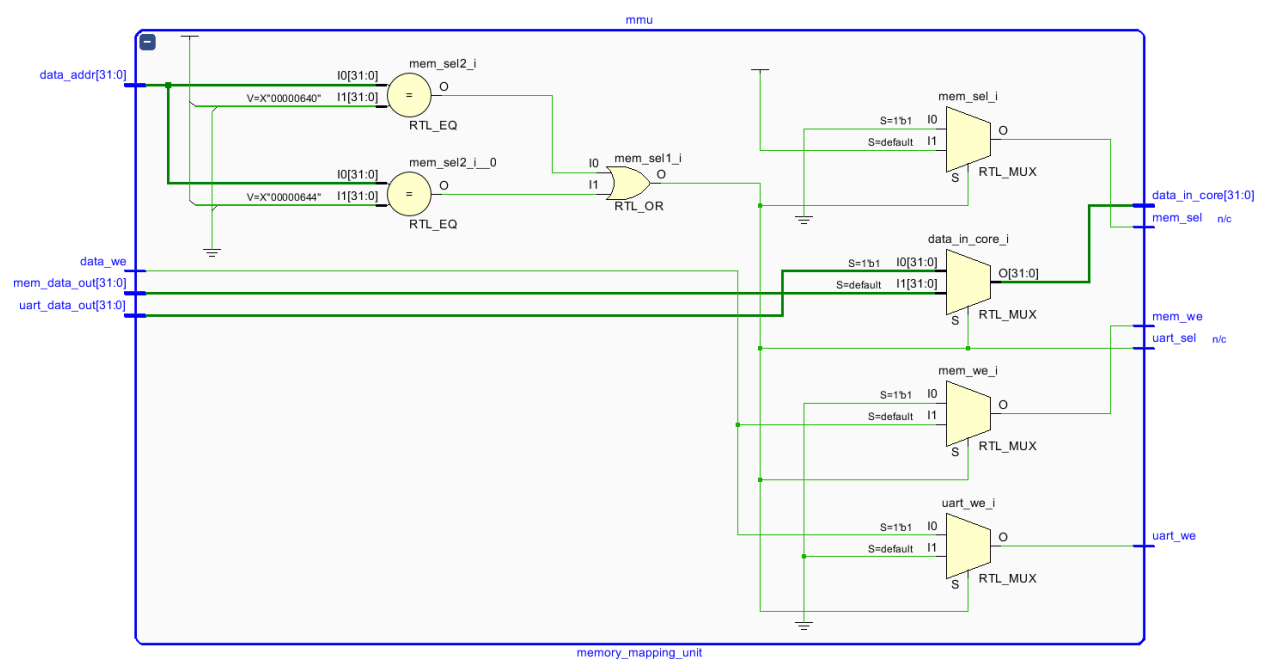
<u>Name</u>	<i><u>Muddassir Ali Siddiqui</u></i>
<u>Instructor</u>	<i><u>Sir Imran & Sir Umer</u></i>
<u>Date</u>	<i><u>8th Sep 2025</u></i>

1. In-Lab Tasks: (Write your lab task & screenshots here)

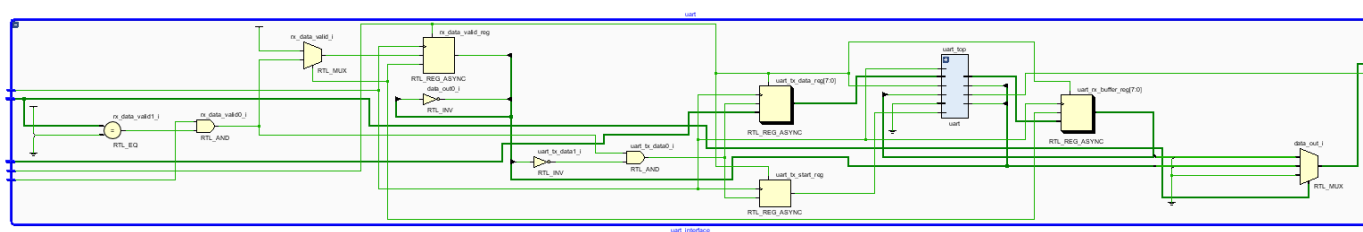
i. Single Cycle Core:



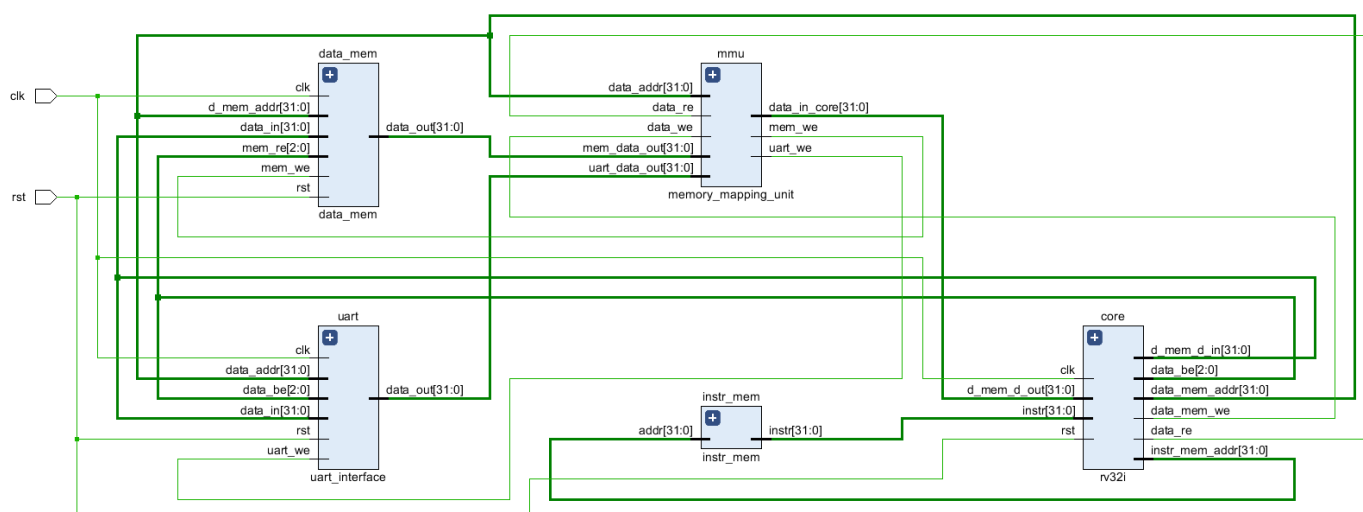
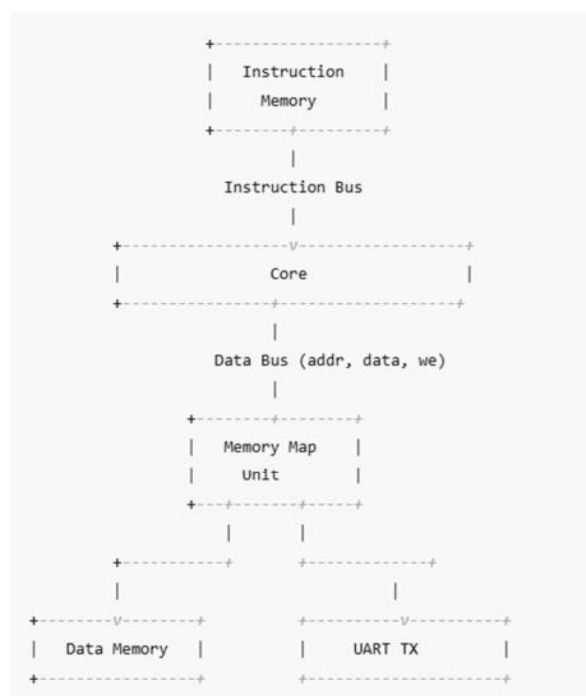
ii. Memory Map Unit:



iii. UART:



iv. SoC Top:



v. Venus Code:

```
1 _start:
2     # Test UART transmission
3     li x5, 1600          # Load UART data address
4     li x6, 0x48          # Load 'H' character
5     sw x6, 0(x5)         # Store to UART (triggers transmission)
6
7     # Check UART status
8     li x5, 1604          # Load UART status address
9 check_status:
10    lw x7, 0(x5)         # Read status register
11    andi x7, x7, 1        # Check busy bit
12    bnez x7, check_status # Loop while busy
13
14    # Send next character
15    li x5, 1600          # Load UART data address
16    li x6, 0x69          # Load 'i' character
17    sw x6, 0(x5)         # Store to UART
18
19    # Continue with more characters...
20
21    # Infinite loop
22 end:
23    j end
```

vi. Verification:

Uart					
> data_addr[31:0]	00000640	00000640	00000048	00000640	
> data_in[31:0]	00000048	00000000	00000048		
uart_we	1				

Uart					
> data_addr[31:0]	00000000	00000069	00000640		
> data_in[31:0]	00000000	00000000	00000069		
uart_we	0				

First, we store the ascii of character 'H' at location of 1600, which means it goes in UART not the Data Memory. So MMU decides where it should go and it is seen that it goes well.

register file																	
we	1																
rs1[4:0]	00																
rs2[4:0]	00																
rd[4:0]	00																
data_in[31:0]	0000002c																
reg_out_1[31:0]	00000000																
reg_out_2[31:0]	00000000																
register_file[0:31][31:0]	00000000,00																
[0][31:0]	00000000																
[1][31:0]	00000000																
[2][31:0]	00000000																
[3][31:0]	00000000																
[4][31:0]	00000000																
[5][31:0]	00000640																
[6][31:0]	00000069																
[7][31:0]	00000001																
[8][31:0]	00000000																

This is our register file when we load and in Venus at the address 1604 there will be empty but in our soc it is present in UART, and it loads successfully.

data_mem[0:1023][31:0]	00000000,00																
[0][31:0]	00000000																
[1][31:0]	00000000																
[2][31:0]	00000000																
[3][31:0]	00000000																
[4][31:0]	00000000																
[5][31:0]	00000000																
[6][31:0]	00000000																
[7][31:0]	00000000																
[8][31:0]	00000000																
[9][31:0]	00000000																
[10][31:0]	00000000																
[11][31:0]	00000000																
[12][31:0]	00000000																
[13][31:0]	00000000																
[14][31:0]	00000000																
[15][31:0]	00000000																
[16][31:0]	00000000																
[17][31:0]	00000000																
[18][31:0]	00000000																
[19][31:0]	00000000																
[20][31:0]	00000000																
[21][31:0]	00000000																

While data memory remain empty according to given assembly code.

2. Critical Analysis: (Write you critical analysis / conclusion here)

In this lab, I integrated a single-cycle RV32I core into an SoC by separating instruction and data memories from the processor and instantiating them in the SoC top. A memory-mapped unit was designed to decode addresses and route load/store operations either to memory or to the UART peripheral. This allowed the processor to communicate with external devices through standard memory-mapped I/O. The modular design improves flexibility, making it easier to extend the SoC with additional peripherals. Overall, the lab demonstrated how a processor, memory, and I/O devices are combined to form a complete SoC.